SQL MINI PROJECT

RETAIL STORE

DATABASE MANAGEMENT SYSTEM

MySQL

SUBMITTED BY:
Manish Giri

SUBMITTED TO:
Career247

DATE OF SUBMISSION:
May 31, 2026

TRANSFORMING DATA INTO BUSINESS INSIGHTS

PROJECT OVERVIEW

Overview

The Retail Store Database Management System is a MySQL-based project developed to simulate and manage the core operations of a retail business. The system is designed to efficiently store, organize, and analyse data related to customers, products, orders, payments, and product reviews within a structured relational database environment.

The project demonstrates the practical implementation of database management concepts and SQL techniques used in real-world business applications. By utilizing a well-structured database design, the system ensures data integrity, consistency, and efficient retrieval of information.

A total of six interconnected tables were created and linked using Primary Keys and Foreign Keys to establish meaningful relationships between business entities. The database was populated with realistic sample data to perform comprehensive analysis and reporting.

The project covers a wide range of SQL operations, including data retrieval, filtering, sorting, aggregation, joins, subqueries, and set operations. These queries were used to generate valuable business insights related to customer purchasing behaviour, product performance, sales trends, inventory management, and payment analysis.

All database development and query execution activities were performed using MySQL Workbench. An Entity Relationship Diagram (ER Diagram) was also developed to visually represent the database structure and relationships among the tables.

PROJECT OBJECTIVES

The primary objective of this project is to design and implement a relational database system capable of managing retail store operations efficiently.

Specific Objectives

• Design a normalized relational database structure.

• Create and manage tables using SQL Data Definition Language (DDL) commands.

• Populate the database with realistic business data using Data Manipulation Language (DML) commands.

• Perform data analysis using SQL queries ranging from basic to advanced levels.

• Generate meaningful business insights from transactional and customer data.

• Understand the practical application of joins, aggregations, subqueries, and set operations.

• Strengthen database management and analytical skills through real-world scenarios.

Project Scope

This project focuses on the management and analysis of retail business data through a structured database system. The scope includes customer management, product management, order processing, payment tracking, product review management, inventory monitoring, and business performance analysis.

The database serves as a foundation for generating reports and insights that support informed decision-making in a retail environment.

DATABASE DESIGN & ARCHITECTURE

3.1 Database Overview

The Retail Store Database Management System was designed using a relational database approach to efficiently manage retail operations. The database stores information related to customers, products, orders, payments, and product reviews while maintaining data integrity through well-defined relationships.

The database was developed in MySQL Workbench and follows normalization principles to minimize redundancy and ensure consistency.

3.2 Tables Used in the Database

The system consists of six major tables:

Customers

Stores customer information including customer ID, name, email address, phone number, and account creation details.

Products

Maintains details about products available for sale, including product name, category, price, stock quantity, and product descriptions.

Orders

Stores order transactions placed by customers. Each order is linked to a customer and contains order date, status, and total amount.

Order Items

Acts as a bridge between orders and products. It stores information about products purchased within each order along with quantity and item price.

Payments

Contains payment-related information such as payment method, payment date, and amount paid for each order.

Product Reviews

Stores customer feedback and ratings for products, helping evaluate customer satisfaction and product performance.

3.3 Database Features

The database was designed with the following features:

- Relational database structure
- Primary Key and Foreign Key implementation
- Data integrity and consistency
- Efficient data retrieval using SQL
- Support for business reporting and analysis
- Scalable architecture for future enhancements

3.4 Constraints Implemented

To maintain data accuracy and consistency, the following constraints were used:

Primary Keys

Each table contains a unique Primary Key to identify individual records.

Foreign Keys

Foreign Keys establish relationships between tables and enforce referential integrity.

NOT NULL Constraint

Prevents important fields from storing empty values.

UNIQUE Constraint

Ensures duplicate values are not entered where uniqueness is required.

AUTO INCREMENT

Automatically generates unique identifiers for new records.

DEFAULT Values

Provides predefined values when no explicit value is supplied.

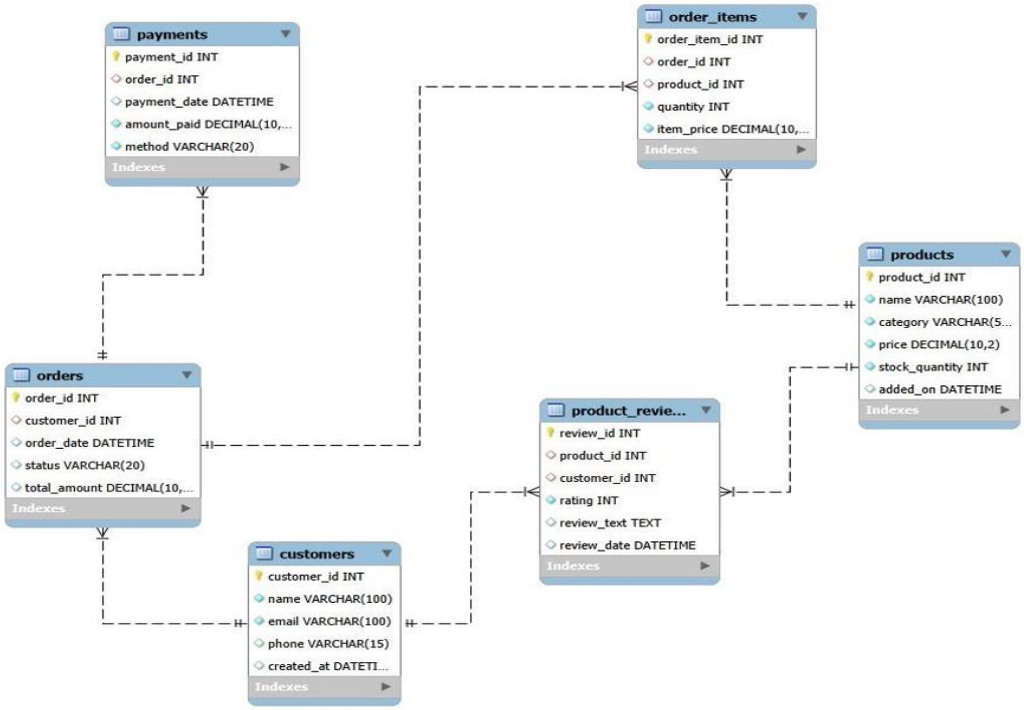
3.5 Database Statistics

Component	Count
Customers	30
Products	50
Orders	400+
Payments	400+
Product Reviews	50
Total Tables	6

The database contains sufficient sample data to perform realistic business analysis and demonstrate various SQL operations.

ENTITY RELATIONSHIP (ER) DIAGRAM

4.1 ER Diagram



4.2 Relationship Summary

The database follows a relational structure where tables are interconnected through Primary Keys and Foreign Keys.

Entity Relationship

Parent Table	Child Table	Relationship Type	Foreign Key	Real-Life Meaning
customers	orders	One-to-Many	orders.customer_id → customers.customer_id	A customer can place multiple orders
orders	order_items	One-to-Many	order_items.order_id → orders.order_id	An order can contain multiple items
products	order_items	One-to-Many	order_items.product_id → products.product_id	A product can appear in many different orders
orders	payments	One-to-Many	payments.order_id → orders.order_id	One or more than one payment per order.
products	product_reviews	One-to-Many	product_reviews.product_id → products.product_id	A product can have many customer reviews
customers	product_reviews	One-to-Many	product_reviews.customer_id → customers.customer_id	A customer can write multiple reviews

4.3 Relationship Explanation

A single customer can place multiple orders, while each order belongs to one customer.

Each order may contain multiple products, and a single product can appear in multiple orders. This relationship is managed through the Order_Items table.

Every payment record is linked to a specific order, enabling payment tracking and financial analysis.

Customers can submit reviews for products they have purchased, allowing the business to evaluate product quality and customer satisfaction.

The ER Diagram provides a visual representation of these relationships and serves as the foundation for efficient database operations and reporting.

1. Create Database & Tables (DDL)

```sql
-- Create Database
CREATE DATABASE IF NOT EXISTS retail_store;
USE retail_store;
```

#	Time	Action	Message
1	07:55:12	CREATE DATABASE IF NOT EXISTS retail_store	1 row(s) affected
2	07:55:12	USE retail_store	0 row(s) affected

```sql
-- 1. Customers Table
CREATE TABLE customers (
    customer_id INT PRIMARY KEY AUTO_INCREMENT,
    name VARCHAR(100) NOT NULL,
    email VARCHAR(100) NOT NULL UNIQUE,
    phone VARCHAR(15),
    created_at DATETIME DEFAULT CURRENT_TIMESTAMP
);
```

3	07:56:09	CREATE TABLE customers (customer_id INT PRIMARY KEY AUTO_INCREMENT, name VARCHAR(100) NOT ...	0 row(s) affected

```sql
-- 2. Products Table
CREATE TABLE products (
    product_id INT PRIMARY KEY AUTO_INCREMENT,
    name VARCHAR(100) NOT NULL,
    category VARCHAR(50) NOT NULL,
    price DECIMAL(10,2) NOT NULL CHECK (price > 0),
    stock_quantity INT NOT NULL DEFAULT 0,
    added_on DATETIME DEFAULT CURRENT_TIMESTAMP
);
```

4	07:56:38	CREATE TABLE products (product_id INT PRIMARY KEY AUTO_INCREMENT, name VARCHAR(100) NOT ...	0 row(s) affected

```sql
-- 3. Orders Table
CREATE TABLE orders (
    order_id INT PRIMARY KEY AUTO_INCREMENT,
    customer_id INT,
    order_date DATETIME DEFAULT CURRENT_TIMESTAMP,
    status VARCHAR(20) DEFAULT 'Pending',
    total_amount DECIMAL(10,2),
    FOREIGN KEY (customer_id) REFERENCES customers(customer_id)
);
```

5	07:57:44	CREATE TABLE orders (order_id INT PRIMARY KEY AUTO_INCREMENT, customer_id INT, order_date D...	0 row(s) affected

```sql
-- 4. Order Items Table
CREATE TABLE order_items (
    order_item_id INT PRIMARY KEY AUTO_INCREMENT,
    order_id INT,
    product_id INT,
    quantity INT NOT NULL CHECK (quantity > 0),
    item_price DECIMAL(10,2) NOT NULL,
    FOREIGN KEY (order_id) REFERENCES orders(order_id),
    FOREIGN KEY (product_id) REFERENCES products(product_id)
);
```

```
6  07:58:39  CREATE TABLE order_items (  order_item_id INT PRIMARY KEY AUTO_INCREMENT,  order_id INT,  product...  0 row(s) affected
```

```sql
-- 5. Payments Table
CREATE TABLE payments (
    payment_id INT PRIMARY KEY AUTO_INCREMENT,
    order_id INT,
    payment_date DATETIME DEFAULT CURRENT_TIMESTAMP,
    amount_paid DECIMAL(10,2) NOT NULL CHECK (amount_paid > 0),
    method VARCHAR(20) NOT NULL,
    FOREIGN KEY (order_id) REFERENCES orders(order_id)
);
```

```
7  07:59:26  CREATE TABLE payments (  payment_id INT PRIMARY KEY AUTO_INCREMENT,  order_id INT,  payment_d...  0 row(s) affected
```

```sql
-- 6. Product Reviews Table
CREATE TABLE product_reviews (
    review_id INT PRIMARY KEY AUTO_INCREMENT,
    product_id INT,
    customer_id INT,
    rating INT NOT NULL CHECK (rating BETWEEN 1 AND 5),
    review_text TEXT,
    review_date DATETIME DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (product_id) REFERENCES products(product_id),
    FOREIGN KEY (customer_id) REFERENCES customers(customer_id)
);
```

```
8  08:00:12  CREATE TABLE product_reviews (  review_id INT PRIMARY KEY AUTO_INCREMENT,  product_id INT,  cus...  0 row(s) affected
```

```sql
SELECT 'All Tables Created Successfully' AS Status;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

Status
All Tables Created Successfully

```
9  08:00:58  SELECT 'All Tables Created Successfully' AS Status LIMIT 0, 1000  1 row(s) returned
```

2. Insert Data

```
-- Customers (with phone trimmed to 15 chars)
INSERT INTO customers (customer_id, name, email, phone, created_at) VALUES (1, 'Thomas Owens', 'user1@example
INSERT INTO customers (customer_id, name, email, phone, created_at) VALUES (2, 'Charles Grant', 'user2@exampl
INSERT INTO customers (customer_id, name, email, phone, created_at) VALUES (3, 'Kaitlin Richards', 'user3@exa
INSERT INTO customers (customer_id, name, email, phone, created_at) VALUES (4, 'Christina Williams', 'user4@e
INSERT INTO customers (customer_id, name, email, phone, created_at) VALUES (5, 'David Allen', 'user5@example.
INSERT INTO customers (customer_id, name, email, phone, created_at) VALUES (6, 'Mark Duke', 'user6@example.co
INSERT INTO customers (customer_id, name, email, phone, created_at) VALUES (7, 'Briana Wright', 'user7@exampl
INSERT INTO customers (customer_id, name, email, phone, created_at) VALUES (8, 'John Bryan', 'user8@example.c
INSERT INTO customers (customer_id, name, email, phone, created_at) VALUES (9, 'Jason Thompson', 'user9@examp
INSERT INTO customers (customer_id, name, email, phone, created_at) VALUES (10, 'Shawn Hill', 'user10@example
INSERT INTO customers (customer_id, name, email, phone, created_at) VALUES (11, 'Walter Jenkins', 'user11@exa
INSERT INTO customers (customer_id, name, email, phone, created_at) VALUES (12, 'Mary Knight', 'user12@exampl
INSERT INTO customers (customer_id, name, email, phone, created_at) VALUES (13, 'Leslie Wilson', 'user13@exam
INSERT INTO customers (customer_id, name, email, phone, created_at) VALUES (14, 'Deborah Arias', 'user14@exam
```

```
2138  08:05:06  INSERT INTO product_reviews (review_id, product_id, customer_id, rating, review_text, review_date) VALUES (48, 4...  1 row(s) affected
2139  08:05:06  INSERT INTO product_reviews (review_id, product_id, customer_id, rating, review_text, review_date) VALUES (49, 1...  1 row(s) affected
2140  08:05:06  INSERT INTO product_reviews (review_id, product_id, customer_id, rating, review_text, review_date) VALUES (50, 3...  1 row(s) affected
```

2.1 After running it, verifying with:

```
2213    SELECT COUNT(*) FROM customers;        -- Should be 30
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

COUNT(*)
30

```
2141  08:06:51  SELECT COUNT(*) FROM customers LIMIT 0, 1000  1 row(s) returned
```

```
2214    SELECT COUNT(*) FROM products;         -- Should be 50
2215
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

COUNT(*)
50

```
2142  08:08:23  SELECT COUNT(*) FROM products LIMIT 0, 1000  1 row(s) returned
```

```
2215    SELECT COUNT(*) FROM orders;           -- Should be around 400
2216
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

COUNT(*)
400

```
2143  08:09:05  SELECT COUNT(*) FROM orders LIMIT 0, 1000  1 row(s) returned
```

LEVEL 1 — Basics

Query: Retrieve customer names and emails for email marketing

```
2217    -- Level 1: Basics
2218
2219    -- 1. Retrieve customer names and emails for email marketing
2220 •  SELECT name, email
2221    FROM customers
2222    ORDER BY name;
```

name	email
Adrienne Green	user27@example.com
Amanda Bright	user19@example.com
Amy Landry	user16@example.com
Austin Flores	user15@example.com
Brandy Wright	user24@example.com
Briana Wright	user7@example.com
Charles Grant	user2@example.com
Christina Williams	user4@example.com
Cindy Hart	user25@example.com
David Allen	user5@example.com
Deborah Arias	user14@example.com
Jason Thompson	user9@example.com
Jeffrey Bray	user18@example.com
Jessica Zamora	user21@example.com
John Bryan	user8@example.com
Joseph Stuart	user28@example.com
Kaitlin Richards	user3@example.com
Kara Zavala	user29@example.com

```
2144  08:11:52  SELECT name, email  FROM customers  ORDER BY name LIMIT 0, 1000   30 row(s) returned
```

- This helps the marketing team extract basic customer contact details for campaigns.

Query: View complete product catalogue with all available details

```
2224    -- 2. View complete product catalog with all available details
2225 •  SELECT * FROM products
2226    ORDER BY product_id;
```

product_id	name	category	price	stock_quantity	added_on
1	Plant No	Home	639.43	152	2024-01-30 06:30:53
2	Population Social	Clothing	4813.68	84	2025-05-30 10:02:50
3	Available Answer	Electronics	2529.51	101	2025-04-13 01:11:46
4	Any Question	Clothing	4759.28	179	2025-06-03 13:34:03
5	Natural Network	Toys	4722.66	75	2023-11-06 00:47:37
6	If Whatever	Electronics	177.40	64	2024-12-19 10:37:14
7	Response Indeed	Clothing	4897.36	36	2025-03-29 02:43:08
8	Every Amount	Home	4173.60	156	2025-04-30 03:11:10
9	Common Study	Toys	985.19	171	2023-07-20 13:06:42
10	Development System	Electronics	4801.78	153	2025-03-12 08:22:57
11	Build Her	Books	1852.64	150	2024-09-08 01:09:15
12	Action Ask	Electronics	4017.01	19	2025-02-14 03:38:06
13	Full West	Books	2112.33	172	2023-09-15 03:13:38
14	Listen Development	Home	296.17	132	2024-10-12 11:49:26
15	Place Low	Electronics	723.97	46	2023-07-05 14:36:07
16	Special Fact	Toys	1094.18	15	2025-03-17 10:42:40
17	Everything Plant	Books	2496.68	120	2023-10-08 20:11:55
18	Some Them	Toys	3673.86	110	2024-07-25 00:33:53

```
2145  08:14:43  SELECT * FROM products  ORDER BY product_id LIMIT 0, 1000   50 row(s) returned
```

- The product manager may want to review all product listings in one go.

Query: List all unique product categories

```
2228    -- 3. List all unique product categories
2229    SELECT DISTINCT category
2230    FROM products
2231    ORDER BY category;
```

category
Books
Clothing
Electronics
Home
Toys

```
2146  15:25:11  SELECT DISTINCT category  FROM products  ORDER BY category LIMIT 0, 1000   5 row(s) returned
```

- Useful for analysing the range of departments or for creating filters on the website.

Query: Show all products priced above ₹1,000

```
2233    -- 4. Show all products priced above ₹1,000
2234    SELECT * FROM products
2235    WHERE price > 1000
2236    ORDER BY price DESC;
```

product_id	name	category	price	stock_quantity	added_on
7	Response Indeed	Clothing	4897.36	36	2025-03-29 02:43:08
2	Population Social	Clothing	4813.68	84	2025-05-30 10:02:50
10	Development System	Electronics	4801.78	153	2025-03-12 08:22:57
4	Any Question	Clothing	4759.28	179	2025-06-03 13:34:03
35	Fire Often	Electronics	4734.89	198	2023-10-22 04:15:51
5	Natural Network	Toys	4722.66	75	2023-11-06 00:47:37
19	Build High	Clothing	4707.14	47	2023-09-01 02:50:01
41	Serious Recognize	Electronics	4523.10	112	2023-08-20 06:16:55
38	Study Total	Toys	4413.68	31	2023-11-08 08:25:07
20	Real Source	Books	4398.66	197	2025-02-08 10:28:27
33	Involve Officer	Clothing	4244.09	112	2024-03-22 11:36:17
48	Group But	Clothing	4180.23	141	2023-11-29 01:38:12
8	Every Amount	Home	4173.60	156	2025-04-30 03:11:10
22	Move Small	Books	4168.08	22	2023-11-12 18:26:30
29	Drop Remember	Electronics	4160.45	89	2023-12-15 12:31:11

```
2147  15:26:58  SELECT * FROM products  WHERE price > 1000  ORDER BY price DESC LIMIT 0, 1000   40 row(s) returned
```

- This helps identify high-value items for premium promotions or pricing strategy reviews.

Query: Display products within a mid-range price bracket (₹2,000 to ₹5,000)

```
2238    -- 5. Display products within a mid-range price bracket (₹2,000 to ₹5,000)
2239    SELECT * FROM products
2240    WHERE price BETWEEN 2000 AND 5000
2241    ORDER BY price DESC;
```

product_id	name	category	price	stock_quantity	added_on
7	Response Indeed	Clothing	4897.36	36	2025-03-29 02:43:08
2	Population Social	Clothing	4813.68	84	2025-05-30 10:02:50
10	Development System	Electronics	4801.78	153	2025-03-12 08:22:57
4	Any Question	Clothing	4759.28	179	2025-06-03 13:34:03
35	Fire Often	Electronics	4734.89	198	2023-10-22 04:15:51
5	Natural Network	Toys	4722.66	75	2023-11-06 00:47:37
19	Build High	Clothing	4707.14	47	2023-09-01 02:50:01
41	Serious Recognize	Electronics	4523.10	112	2023-08-20 06:16:55
38	Study Total	Toys	4413.68	31	2023-11-08 08:25:07
20	Real Source	Books	4398.66	197	2025-02-08 10:28:27
33	Involve Officer	Clothing	4244.09	112	2024-03-22 11:36:17
48	Group But	Clothing	4180.23	141	2023-11-29 01:38:12
8	Every Amount	Home	4173.60	156	2025-04-30 03:11:10
22	Move Small	Books	4168.08	22	2023-11-12 18:26:30
29	Drop Remember	Electronics	4160.45	89	2023-12-15 12:31:11
39	Data Add	Books	4063.38	62	2025-05-26 23:39:16

```
2148  15:49:24  SELECT * FROM products  WHERE price BETWEEN 2000 AND 5000  ORDER BY price DESC LIMIT 0, 1000   34 row(s) returned
```

- A merchandising team might need this to create a mid-tier pricing campaign.

Query: Fetch data for specific customer IDs (e.g., from loyalty program list)

```
2243    -- 6. Fetch data for specific customer IDs (e.g., from loyalty program)
2244    SELECT * FROM customers
2245    WHERE customer_id IN (1, 5, 10, 15, 20);
```

customer_id	name	email	phone	created_at
1	Thomas Owens	user1@example.com	142-479-1945	2024-10-14 16:01:12
5	David Allen	user5@example.com	(751)456-8289x1	2023-10-29 02:43:00
10	Shawn Hill	user10@example.com	(268)113-3152x7	2023-12-14 20:46:43
15	Austin Flores	user15@example.com	329.901.1576x66	2024-06-13 09:03:42
20	Megan Lee	user20@example.com	7044478333	2024-07-09 16:13:14
NULL	NULL	NULL	NULL	NULL

```
2149  15:52:07  SELECT * FROM customers  WHERE customer_id IN (1, 5, 10, 15, 20) LIMIT 0, 1000   5 row(s) returned
```

- This is used when customer IDs are pre-selected from another system.

Query: Identify customers whose names start with the letter 'A'

```
2247    -- 7. Identify customers whose names start with the letter 'A'
2248    SELECT name, email, phone
2249    FROM customers
2250    WHERE name LIKE 'A%'
2251    ORDER BY name;
```

name	email	phone
Adrienne Green	user27@example.com	530.644.8455x93
Amanda Bright	user19@example.com	380.981.9798x69
Amy Landry	user16@example.com	+1-278-019-3748
Austin Flores	user15@example.com	329.901.1576x66

```
2150  15:53:57  SELECT name, email, phone  FROM customers  WHERE name LIKE 'A%'  ORDER BY name LIMIT 0, 1000   4 row(s) returned
```

- Used for alphabetical segmentation in outreach or app display.

Query: List electronics products priced under ₹3,000

```
2253    -- 8. List Electronics products priced under ₹3,000
2254    SELECT product_id, name, price, stock_quantity
2255    FROM products
2256    WHERE category = 'Electronics' AND price < 3000
2257    ORDER BY price DESC;
```

product_id	name	price	stock_quantity
3	Available Answer	2529.51	101
31	Series Page	2070.37	83
34	Despite Win	1340.34	64
15	Place Low	723.97	46
47	Southern Thing	512.46	40
44	Actually Term	396.11	85
6	If Whatever	177.40	64
NULL	NULL	NULL	NULL

```
2151  16:02:54  SELECT product_id, name, price, stock_quantity  FROM products  WHERE category = 'Electronics' AND price < 3000  ORDE...   7 row(s) returned
```

- Used by merchandising or frontend teams to showcase budget electronics.

Query: Display product names and prices in descending order of price

```
2259    -- 9. Display product names and prices in descending order of price
2260 •  SELECT name, price
2261    FROM products
2262    ORDER BY price DESC, name;
```

name	price
Response Indeed	4897.36
Population Social	4813.68
Development System	4801.78
Any Question	4759.28
Fire Often	4734.89
Natural Network	4722.66
Build High	4707.14
Serious Recognize	4523.10
Study Total	4413.68
Real Source	4398.66
Involve Officer	4244.09
Group But	4180.23
Every Amount	4173.60
Move Small	4168.08
Drop Remember	4160.45
Data Add	4063.38

```
2152  16:09:36  SELECT name, price  FROM products  ORDER BY price DESC, name LIMIT 0, 1000    50 row(s) returned
```

- This helps teams easily view and compare top-priced items.

Query: Display product names and prices, sorted by price and then by name

```
2264    -- 10. Display product names and prices, sorted by price then by name
2265 •  SELECT name, category, price
2266    FROM products
2267    ORDER BY price DESC, name ASC;
```

name	category	price
Response Indeed	Clothing	4897.36
Population Social	Clothing	4813.68
Development System	Electronics	4801.78
Any Question	Clothing	4759.28
Fire Often	Electronics	4734.89
Natural Network	Toys	4722.66
Build High	Clothing	4707.14
Serious Recognize	Electronics	4523.10
Study Total	Toys	4413.68
Real Source	Books	4398.66
Involve Officer	Clothing	4244.09
Group But	Clothing	4180.23
Every Amount	Home	4173.60
Move Small	Books	4168.08
Drop Remember	Electronics	4160.45
Data Add	Books	4063.38

```
2153  16:11:54  SELECT name, category, price  FROM products  ORDER BY price DESC, name ASC LIMIT 0, 1000    50 row(s) returned
```

- The merchandising or catalogue team may want to list products from most expensive to cheapest. If multiple products have the same price, they should be sorted alphabetically for clarity on storefronts or printed catalogues.

LEVEL 2 — Filtering & Formatting

Query: Retrieve orders where customer information is missing (possibly due to data migration or deletion)

```
2272    -- 1. Retrieve orders where customer information is missing
2273 •  SELECT *
2274    FROM orders
2275    WHERE customer_id IS NULL;
```

order_id	customer_id	order_date	status	total_amount
NULL	NULL	NULL	NULL	NULL

```
2154  16:21:23  SELECT * FROM orders WHERE customer_id IS NULL LIMIT 0, 1000   0 row(s) returned
```

- Used to identify orphaned orders or test data where customer_id is not linked.

Query: Display customer names and emails using column aliases for frontend readability

```
2277    -- 2. Display customer names and emails with column aliases
2278 •  SELECT
2279        name AS customer_name,
2280        email AS customer_email,
2281        phone AS contact_number
2282    FROM customers
2283    ORDER BY customer_name;
```

customer_name	customer_email	contact_number
Adrienne Green	user27@example.com	530.644.8455x93
Amanda Bright	user19@example.com	380.981.9798x69
Amy Landry	user16@example.com	+1-278-019-3748
Austin Flores	user15@example.com	329.901.1576x66
Brandy Wright	user24@example.com	853.520.8915x61
Briana Wright	user7@example.com	223-833-9635
Charles Grant	user2@example.com	9153947511
Christina Williams	user4@example.com	586-605-5061x06
Cindy Hart	user25@example.com	617.574.8421x41
David Allen	user5@example.com	(751)456-8289x1
Deborah Arias	user14@example.com	811-821-2144x97
Jason Thompson	user9@example.com	1862659420
Jeffrey Bray	user18@example.com	164.962.0222x92
Jessica Zamora	user21@example.com	177-816-7773x00
John Bryan	user8@example.com	045.568.0798x27
Joseph Stuart	user28@example.com	363-045-4287

```
2155  16:40:40  SELECT  name AS customer_name,  email AS customer_email,  phone AS contact_number FROM customers  ORDER B...  30 row(s) returned
```

- Useful for feeding into frontend displays or report headings that require user-friendly labels.

Query: Calculate total value per item ordered by multiplying quantity and item price

```sql
-- 3. Calculate total value per order item (quantity * item_price)
SELECT
    oi.order_item_id,
    oi.order_id,
    p.name AS product_name,
    oi.quantity,
    oi.item_price,
    (oi.quantity * oi.item_price) AS total_value
FROM order_items oi
JOIN products p ON oi.product_id = p.product_id
ORDER BY total_value DESC;
```

order_item_id	order_id	product_name	quantity	item_price	total_value
8	6	Response Indeed	3	4897.36	14692.08
16	9	Response Indeed	3	4897.36	14692.08
21	10	Response Indeed	3	4897.36	14692.08
233	78	Response Indeed	3	4897.36	14692.08
939	315	Response Indeed	3	4897.36	14692.08
179	63	Population Social	3	4813.68	14441.04
323	107	Population Social	3	4813.68	14441.04
459	157	Population Social	3	4813.68	14441.04
822	281	Population Social	3	4813.68	14441.04
914	308	Population Social	3	4813.68	14441.04
13	8	Development Sy...	3	4801.78	14405.34
86	32	Development Sy...	3	4801.78	14405.34
157	55	Development Sy...	3	4801.78	14405.34
200	68	Development Sy...	3	4801.78	14405.34
332	110	Development Sy...	3	4801.78	14405.34
500	174	Development Sy...	3	4801.78	14405.34

2156 16:51:42 SELECT oi.order_item_id, oi.order_id, p.name AS product_name, oi.quantity, oi.item_price, (oi.quantity * oi.item_... 1000 row(s) returned

- This can help generate per-line item bill details or invoice breakdowns.

Query: Combine customer name and phone number in a single column

```sql
-- 4. Combine customer name and phone in one column
SELECT
    name,
    CONCAT(name, ' | ', COALESCE(phone, 'No Phone')) AS full_contact_info
FROM customers;
```

name	full_contact_info
Thomas Owens	Thomas Owens | 142-479-1945
Charles Grant	Charles Grant | 9153947511
Kaitlin Richards	Kaitlin Richards | 2073473421
Christina Williams	Christina Williams | 586-605-5061x06
David Allen	David Allen | (751)456-8289x1
Mark Duke	Mark Duke | (144)957-2811
Briana Wright	Briana Wright | 223-833-9635
John Bryan	John Bryan | 045.568.0798x27
Jason Thompson	Jason Thompson | 1862659420
Shawn Hill	Shawn Hill | (268)113-3152x7
Walter Jenkins	Walter Jenkins | 536-329-0817x71
Mary Knight	Mary Knight | 361-636-3802
Leslie Wilson	Leslie Wilson | +1-256-261-1984
Deborah Arias	Deborah Arias | 811-821-2144x97
Austin Flores	Austin Flores | 329.901.1576x66
Amy Landry	Amy Landry | +1-278-019-3748

Result 18

```
2157  16:57:00  SELECT  name,  CONCAT(name, ' | ', COALESCE(phone, 'No Phone')) AS full_contact_info FROM customers LIMIT 0, 1000  30 row(s) returned
```

- Used to show brief customer summaries or contact lists.

Query: Extract only the date part from order timestamps for date-wise reporting

```
2303    -- 5. Extract only date from order_date
2304    SELECT
2305        order_id,
2306        customer_id,
2307        DATE(order_date) AS order_date_only,
2308        status,
2309        total_amount
2310    FROM orders
2311    ORDER BY order_date_only DESC;
```

order_id	customer_id	order_date_only	status	total_amount
62	10	2025-06-13	Cancelled	35723.17
173	19	2025-06-13	Shipped	6875.62
286	15	2025-06-11	Pending	26952.09
308	14	2025-06-11	Cancelled	36411.35
356	9	2025-06-11	Pending	11584.94
9	6	2025-06-10	Pending	24219.20
127	1	2025-06-10	Pending	20160.64
134	3	2025-06-09	Pending	15483.53
166	14	2025-06-09	Pending	14925.84
313	25	2025-06-09	Cancelled	24245.67
256	17	2025-06-08	Delivered	40944.10
129	29	2025-06-06	Delivered	30752.72
231	14	2025-06-06	Cancelled	27402.85
248	2	2025-06-06	Pending	9829.50
298	25	2025-06-06	Pending	20011.26
390	7	2025-06-06	Delivered	18092.48

Result 19

```
2158  17:04:59  SELECT  order_id,  customer_id,  DATE(order_date) AS order_date_only,  status,  total_amount FROM orders  ORDE...  400 row(s) returned
```

- Helps group or filter orders by date without considering time.

Query: List products that do not have any stock left

```
2313    -- 6. List products with zero stock (Out of Stock)
2314    SELECT product_id, name, category, price
2315    FROM products
2316    WHERE stock_quantity = 0
2317    ORDER BY category;
```

product_id	name	category	price
NULL	NULL	NULL	NULL

```
2159  17:10:49  SELECT product_id, name, category, price  FROM products  WHERE stock_quantity = 0  ORDER BY category LIMIT 0, 1000  0 row(s) returned
```

- This helps the inventory team identify out-of-stock items.

Query: Show orders with their total amount formatted

```
2319    -- 7. Show orders with their total amount formatted
2320 •  SELECT
2321        order_id,
2322        customer_id,
2323        DATE(order_date) AS order_date,
2324        status,
2325        CONCAT('₹', FORMAT(total_amount, 2)) AS formatted_total
2326    FROM orders
2327    ORDER BY total_amount DESC
2328    LIMIT 10;
```

order_id	customer_id	order_date	status	formatted_total
107	14	2025-02-01	Shipped	₹48,042.06
393	28	2024-12-17	Pending	₹44,504.56
309	29	2024-09-13	Delivered	₹43,867.08
315	2	2025-01-27	Cancelled	₹42,056.04
225	29	2024-10-04	Delivered	₹41,885.52
281	3	2024-11-08	Shipped	₹41,679.11
300	29	2025-05-04	Delivered	₹41,322.58
223	2	2024-11-25	Cancelled	₹41,020.75
256	17	2025-06-08	Delivered	₹40,944.10
189	25	2024-12-17	Shipped	₹40,872.29

```
2160  17:16:53  SELECT  order_id,  customer_id,  DATE(order_date) AS order_date,  status,  CONCAT('₹', FORMAT(total_amount, 2))...  10 row(s) returned
```

LEVEL 3 — Aggregations

Query: Count the total number of orders placed

```
2330    -- Level 3: Aggregations & GROUP BY
2331
2332    -- 1. Total number of orders
2333 •  SELECT COUNT(*) AS total_orders FROM orders;
```

total_orders
400

```
2161  17:23:27  SELECT COUNT(*) AS total_orders FROM orders LIMIT 0, 1000  1 row(s) returned
```

- Used by business managers to track order volume over time.

Query: Calculate the total revenue collected from all

```
2335    -- 2. Total revenue from all orders
2336 •  SELECT ROUND(SUM(total_amount), 2) AS total_revenue FROM orders;
```

total_revenue
6960973.66

2162 17:27:22 SELECT ROUND(SUM(total_amount), 2) AS total_revenue FROM orders LIMIT 0, 1000 1 row(s) returned

- This gives the overall sales value.

Query: Calculate the average order value

```
2338    -- 3. Average order value
2339 •  SELECT ROUND(AVG(total_amount), 2) AS average_order_value FROM orders;
```

average_order_value
17402.43

2163 17:31:52 SELECT ROUND(AVG(total_amount), 2) AS average_order_value FROM orders LIMIT 0, 1000 1 row(s) returned

- Used for understanding customer spending patterns.

Query: Count the number of customers who have placed at least one order

```
2341    -- 4. Number of unique customers who placed orders
2342 •  SELECT COUNT(DISTINCT customer_id) AS active_customers FROM orders;
```

active_customers
30

2164 17:43:31 SELECT COUNT(DISTINCT customer_id) AS active_customers FROM orders LIMIT 0, 1000 1 row(s) returned

- This identifies active customers.

Query: Find the number of orders placed by each customer

```
2344    -- 5. Number of orders per customer (Top customers)
2345 •  SELECT
2346        c.name,
2347        COUNT(o.order_id) AS order_count,
2348        ROUND(SUM(o.total_amount), 2) AS total_spent
2349    FROM orders o
2350    JOIN customers c ON o.customer_id = c.customer_id
2351    GROUP BY c.customer_id, c.name
2352    ORDER BY order_count DESC, total_spent DESC
2353    LIMIT 10;
```

name	order_count	total_spent
Kara Zavala	20	435408.89
Joseph Stuart	17	362584.19
Charles Grant	17	284420.07
Randy Mooney	17	262189.71
Kaitlin Richards	17	253783.31
Megan Lee	17	229435.29
Victoria Acevedo	16	281841.38
Amy Landry	16	278469.73
Mark Duke	16	212173.58
Brandy Wright	15	379286.82

```
2165  17:59:37  SELECT  c.name,  COUNT(o.order_id) AS order_count,  ROUND(SUM(o.total_amount), 2) AS total_spent FROM orders ...  10 row(s) returned
```

```sql
select customer_id, count(*) as total_orders
from orders
group by customer_id;
```

customer_id	total_orders
1	12
2	17
3	17
4	9
5	14
6	16
7	13
8	10
9	10
10	13
11	9
12	11
13	12
14	15
15	15
16	16

Result 27

```
2166  18:01:28  select customer_id, count(*) as total_orders from orders group by customer_id LIMIT 0, 1000  30 row(s) returned
```

- Helpful for identifying top or repeat customers.

Query: Find total sales amount made by each customer

```sql
-- 6. Total sales amount by each customer
SELECT
    customer_id,
    COUNT(order_id) AS total_orders,
    ROUND(SUM(total_amount), 2) AS total_sales
FROM orders
GROUP BY customer_id
ORDER BY total_sales DESC;
```

customer_id	total_orders	total_sales
29	20	435408.89
24	15	379286.82
28	17	362584.19
14	15	360324.18
25	15	335100.94
2	17	284420.07
30	16	281841.38
16	16	278469.73
5	14	262504.19
17	17	262189.71
15	15	260499.91
3	17	253783.31
10	13	252722.75
22	15	237879.99
20	17	229435.29
19	15	222256.24

Result 28 ×

2167 18:03:49 SELECT customer_id, COUNT(order_id) AS total_orders, ROUND(SUM(total_amount), 2) AS total_sales FROM orders ... 30 row(s) returned

Query: List the number of products sold per category

```
2368    -- 7. Number of products per category
2369 •  SELECT
2370        category,
2371        COUNT(*) AS product_count,
2372        ROUND(AVG(price), 2) AS avg_price
2373    FROM products
2374    GROUP BY category
2375    ORDER BY product_count DESC;
```

category	product_count	avg_price
Electronics	14	2653.39
Clothing	13	3434.58
Home	8	2146.37
Toys	8	2516.53
Books	7	3167.08

2168 18:06:12 SELECT category, COUNT(*) AS product_count, ROUND(AVG(price), 2) AS avg_price FROM products GROUP BY c... 5 row(s) returned

- This helps category managers assess performance by department.

Query: Find the average item price per category

```
2377    -- 8. Average item price per category
2378 •  SELECT category, avg(price) as avg_price
2379    FROM products
2380    GROUP BY category;
```

	category	avg_price
▶	Home	2146.367500
	Clothing	3434.581538
	Electronics	2653.388571
	Toys	2516.526250
	Books	3167.084286

```
2169  18:19:40  SELECT category, avg(price) as avg_price FROM products GROUP BY category LIMIT 0, 1000   5 row(s) returned
```

- Useful to compare pricing across departments.

Query: Show number of orders placed per day

```
2382    -- 9. Number of orders placed per day
2383 •  SELECT DATE(order_date) AS order_day, count(order_id) orders_per_day
2384    FROM orders
2385    GROUP BY DATE(order_date);
```

	order_day	orders_per_day
▶	2025-03-02	3
	2024-10-09	2
	2025-05-08	2
	2024-09-19	3
	2025-04-08	3
	2024-10-25	3
	2024-07-29	2
	2024-07-30	2
	2025-06-10	2
	2025-02-16	2
	2025-03-11	3
	2025-01-15	4
	2025-05-12	2
	2024-09-24	1
	2024-07-18	1
	2024-09-09	3

Result 31 ×

```
2170  18:25:09  SELECT DATE(order_date) AS order_day, count(order_id) orders_per_day FROM orders GROUP BY DATE(order_date) LIMIT ...   232 row(s) returned
```

- Used to track daily business activity and demand trends.

Query: List total payments received per payment method

```
2387    -- 10. Total revenue per payment method
2388 •  SELECT
2389        method,
2390        COUNT(*) AS transaction_count,
2391        ROUND(SUM(amount_paid), 2) AS total_amount
2392    FROM payments
2393    GROUP BY method
2394    ORDER BY total_amount DESC;
```

method	transaction_count	total_amount
Debit Card	113	1930577.88
Credit Card	96	1754603.10
Net Banking	98	1658383.90
UPI	93	1617408.78

```
2171  18:30:21  SELECT  method,  COUNT(*) AS transaction_count,  ROUND(SUM(amount_paid), 2) AS total_amount FROM payments  ...  4 row(s) returned
```

- Helps the finance team understand preferred transaction modes.

LEVEL 4 — Multi-Table Queries (JOINS)

Query: Retrieve order details along with the customer name (INNER JOIN)

```
2388    -- Level 4: Multi-Table JOINS
2389
2390    -- 1. Orders with customer names
2391    SELECT
2392        o.order_id,
2393        c.name AS customer_name,
2394        o.order_date,
2395        o.status,
2396        o.total_amount
2397    FROM orders o
2398    JOIN customers c ON o.customer_id = c.customer_id
2399    ORDER BY o.order_date DESC;
```

order_id	customer_name	order_date	status	total_amount
62	Shawn Hill	2025-06-13 23:21:01	Cancelled	35723.17
173	Amanda Bright	2025-06-13 10:58:21	Shipped	6875.62
356	Jason Thompson	2025-06-11 13:30:19	Pending	11584.94
286	Austin Flores	2025-06-11 08:36:00	Pending	26952.09
308	Deborah Arias	2025-06-11 03:03:58	Cancelled	36411.35
127	Thomas Owens	2025-06-10 18:01:32	Pending	20160.64
9	Mark Duke	2025-06-10 17:00:25	Pending	24219.20
313	Cindy Hart	2025-06-09 19:00:34	Cancelled	24245.67
134	Kaitlin Richards	2025-06-09 13:59:27	Pending	15483.53
166	Deborah Arias	2025-06-09 06:26:55	Pending	14925.84
256	Randy Mooney	2025-06-08 16:15:49	Delivered	40944.10
129	Kara Zavala	2025-06-06 21:11:17	Delivered	30752.72
231	Deborah Arias	2025-06-06 18:09:21	Cancelled	27402.85
390	Briana Wright	2025-06-06 16:22:38	Delivered	18092.48
248	Charles Grant	2025-06-06 14:36:15	Pending	9829.50

Result 33

```
2172  18:37:04  SELECT  o.order_id,  c.name AS customer_name,  o.order_date,  o.status,  o.total_amount FROM orders o JOIN cust...  400 row(s) returned
```

```sql
SELECT
    o.order_id,
    c.name AS customer_name,
    o.order_date,
    o.status,
    o.total_amount
FROM orders o
JOIN customers c ON o.customer_id = c.customer_id
```

order_id	customer_name	order_date	status	total_amount
14	Thomas Owens	2024-09-24 21:21:38	Shipped	15803.34
17	Thomas Owens	2024-08-19 21:17:57	Pending	11173.77
61	Thomas Owens	2024-12-26 23:21:58	Shipped	13053.00
76	Thomas Owens	2025-01-14 22:59:54	Pending	23506.81
92	Thomas Owens	2024-09-26 11:33:58	Shipped	834.75
109	Thomas Owens	2025-03-17 04:56:18	Shipped	12190.14
127	Thomas Owens	2025-06-10 18:01:32	Pending	20160.64
135	Thomas Owens	2025-03-11 22:58:23	Pending	8891.79
144	Thomas Owens	2024-09-19 09:18:22	Shipped	12093.54
221	Thomas Owens	2024-09-25 05:49:39	Pending	12437.61
278	Thomas Owens	2025-02-01 20:05:38	Delivered	32015.16
379	Thomas Owens	2025-03-22 15:57:58	Cancelled	21586.89
56	Charles Grant	2024-06-23 15:59:57	Pending	6828.61
169	Charles Grant	2025-02-11 04:04:38	Cancelled	9426.30
209	Charles Grant	2024-10-16 10:07:48	Shipped	7690.62
223	Charles Grant	2024-11-25 01:32:18	Cancelled	41020.75

Result 34

```
2173 18:41:40 SELECT o.order_id, c.name AS customer_name, o.order_date, o.status, o.total_amount FROM orders o JOIN cust... 400 row(s) returned
```

- Used for displaying which customer placed each order.

Query: Get list of products that have been sold (INNER JOIN with order_items)

```sql
-- 2. Products that have been sold with quantities
SELECT
    p.name,
    p.category,
    SUM(oi.quantity) AS total_sold,
    p.stock_quantity AS current_stock
FROM products p
LEFT JOIN order_items oi ON p.product_id = oi.product_id
GROUP BY p.product_id, p.name, p.category, p.stock_quantity
ORDER BY total_sold DESC;
```

name	category	total_sold	current_stock
Assume Serve	Home	83	10
Age Treatment	Home	67	160
Some Them	Toys	63	110
Southern Thing	Electronics	62	40
Every Amount	Home	61	156
Move Small	Books	60	22
Real Source	Books	60	197
Between Up	Toys	58	61
Everything Plant	Books	58	120
Group But	Clothing	58	141
Fire Often	Electronics	58	198
Common Study	Toys	57	171
Series Page	Electronics	54	83
If Whatever	Electronics	54	64
Least Green	Toys	54	81
Development S...	Electronics	53	153

Result 35

```
2174 18:52:23 SELECT p.name, p.category, SUM(oi.quantity) AS total_sold, p.stock_quantity AS current_stock FROM products p L... 50 row(s) returned
```

- Used to find which products were actually included in orders.

Query: List all orders with their payment method (INNER JOIN)

```
2421    -- 3. Orders with payment details
2422    SELECT
2423        o.order_id,
2424        c.name,
2425        o.total_amount,
2426        p.method,
2427        p.amount_paid,
2428        p.payment_date
2429    FROM orders o
2430    JOIN customers c ON o.customer_id = c.customer_id
2431    JOIN payments p ON o.order_id = p.order_id
2432    ORDER BY o.order_id;
```

order_id	name	total_amount	method	amount_paid	payment_date
1	Megan Lee	9414.28	Credit Card	9414.28	2025-03-02 08:16:11
2	Jeffrey Bray	532.20	Net Banking	532.20	2024-10-09 19:00:21
3	Austin Flores	5164.56	Credit Card	5164.56	2025-05-08 00:55:27
4	Walter Jenkins	9469.78	UPI	9469.78	2024-09-19 22:28:13
5	Mary Knight	14501.86	UPI	14501.86	2025-04-08 18:26:06
6	Kara Zavala	31050.17	UPI	31050.17	2024-10-25 08:16:59
7	Peter Phillips	3043.67	UPI	3043.67	2024-07-29 12:45:47
8	Amanda Bright	32714.06	Net Banking	32714.06	2024-07-30 23:42:49
9	Mark Duke	24219.20	Net Banking	24219.20	2025-06-10 17:36:25
10	Joseph Stuart	24342.52	Debit Card	24342.52	2025-02-16 13:43:59
11	Cindy Hart	16196.13	Credit Card	16196.13	2025-03-11 15:05:46
12	Joseph Stuart	9890.72	Credit Card	9890.72	2025-01-15 14:00:53
13	Cindy Hart	9627.36	UPI	9627.36	2025-05-12 14:01:29
14	Thomas Owens	15803.34	UPI	15803.34	2024-09-24 22:11:38
15	Kara Zavala	12421.88	UPI	12421.88	2024-07-18 03:45:24
16	David Allen	22856.73	UPI	22856.73	2024-09-09 03:38:40

Result 36

```
2175 18:56:56 SELECT o.order_id, c.name, o.total_amount, p.method, p.amount_paid, p.payment_date FROM orders o JOIN ... 400 row(s) returned
```

- Used by finance or audit teams to see how each order was paid for.

Query: Get list of customers and their orders (LEFT JOIN)

```
2434    -- 4. All customers and their orders (including those with no orders)
2435 •  SELECT
2436        c.name,
2437        COUNT(o.order_id) AS total_orders,
2438        ROUND(SUM(o.total_amount), 2) AS total_spent
2439    FROM customers c
2440    LEFT JOIN orders o ON c.customer_id = o.customer_id
2441    GROUP BY c.customer_id, c.name
2442    ORDER BY total_spent DESC;
```

name	total_orders	total_spent
Kara Zavala	20	435408.89
Brandy Wright	15	379286.82
Joseph Stuart	17	362584.19
Deborah Arias	15	360324.18
Cindy Hart	15	335100.94
Charles Grant	17	284420.07
Victoria Acevedo	16	281841.38
Amy Landry	16	278469.73
David Allen	14	262504.19
Randy Mooney	17	262189.71
Austin Flores	15	260499.91
Kaitlin Richards	17	253783.31
Shawn Hill	13	252722.75
Peter Phillips	15	237879.99
Megan Lee	17	229435.29
Amanda Bright	15	222256.24

Result 37 ×

```
2176  19:01:45  SELECT  c.name,  COUNT(o.order_id) AS total_orders,  ROUND(SUM(o.total_amount), 2) AS total_spent FROM custom...  30 row(s) returned
```

```
2444 •  SELECT
2445    c.customer_id,
2446    c.name,o.order_id
2447    FROM customers c
2448    LEFT JOIN orders o ON c.customer_id = o.customer_id
```

customer_id	name	order_id
1	Thomas Owens	14
1	Thomas Owens	17
1	Thomas Owens	61
1	Thomas Owens	76
1	Thomas Owens	92
1	Thomas Owens	109
1	Thomas Owens	127
1	Thomas Owens	135
1	Thomas Owens	144
1	Thomas Owens	221
1	Thomas Owens	278
1	Thomas Owens	379
2	Charles Grant	56
2	Charles Grant	169
2	Charles Grant	209
2	Charles Grant	223

Result 38 ×

```
2177  19:05:15  SELECT c.customer_id, c.name,o.order_id FROM customers c LEFT JOIN orders o ON c.customer_id = o.customer_id LIMIT 0,...  400 row(s) returned
```

- Used to find all customers and see who has or hasn't placed orders.

Query: List all products along with order item quantity (LEFT JOIN)

```
2450    -- 5. All products along with order item quantity
2451 •  SELECT p.name AS product_name, ot.quantity
2452    FROM products AS p
2453    LEFT JOIN order_items AS ot ON p.product_id = ot.product_id
```

product_name	quantity
Plant No	1
Plant No	3
Plant No	1
Plant No	1
Plant No	2
Plant No	3
Plant No	2
Plant No	2
Plant No	2
Plant No	2
Plant No	2
Plant No	3
Plant No	3
Plant No	2
Plant No	2
Plant No	2

Result 39

2178 19:11:24 SELECT p.name AS product_name, ot.quantity FROM products AS p LEFT JOIN order_items AS ot ON p.product_id = ot.produ... 1000 row(s) returned

- Useful for inventory teams to track what sold and what hasn't.

Query: List all payments including those with no matching orders (RIGHT JOIN)

```
2455    -- 6. All payments including those with no matching orders
2456 •  SELECT o.order_id, p.payment_id, p.method, p.amount_paid
2457    FROM orders AS o
2458    RIGHT JOIN payments AS p ON o.order_id = p.order_id;
```

order_id	payment_id	method	amount_paid
1	1	Credit Card	9414.28
2	2	Net Banking	532.20
3	3	Credit Card	5164.56
4	4	UPI	9469.78
5	5	UPI	14501.86
6	6	UPI	31050.17
7	7	UPI	3043.67
8	8	Net Banking	32714.06
9	9	Net Banking	24219.20
10	10	Debit Card	24342.52
11	11	Credit Card	16196.13
12	12	Credit Card	9890.72
13	13	UPI	9627.36
14	14	UPI	15803.34
15	15	UPI	12421.88
16	16	UPI	22856.73

Result 40

2179 19:18:30 SELECT o.order_id, p.payment_id, p.method, p.amount_paid FROM orders AS o RIGHT JOIN payments AS p ON o.order_id = ... 400 row(s) returned

- Rare but used when ensuring all payments are mapped correctly.

Query: Combine data from three tables: customer, order, and payment

```
2460    -- 7. Combine data from three tables: customer, order & payment
2461 •  SELECT c.name AS customer_name,
2462           c.email AS email_address,
2463           o.order_id,
2464           o.order_date,
2465           p.payment_date,
2466           p.amount_paid,
2467           p.method
2468    FROM customers AS c
2469    JOIN orders AS o ON c.customer_id = o.customer_id
2470    JOIN payments AS p ON p.order_id = o.order_id;
```

customer_name	email_address	order_id	order_date	payment_date	amount_paid	method
Thomas Owens	user1@example.com	14	2024-09-24 21:21:38	2024-09-24 22:11:38	15803.34	UPI
Thomas Owens	user1@example.com	17	2024-08-19 21:17:57	2024-08-19 21:49:57	11173.77	UPI
Thomas Owens	user1@example.com	61	2024-12-26 23:21:58	2024-12-26 23:31:58	13053.00	Net Banking
Thomas Owens	user1@example.com	76	2025-01-14 22:59:54	2025-01-14 23:33:54	23506.81	Credit Card
Thomas Owens	user1@example.com	92	2024-09-26 11:33:58	2024-09-26 12:03:58	834.75	Net Banking
Thomas Owens	user1@example.com	109	2025-03-17 04:56:18	2025-03-17 05:16:18	12190.14	UPI
Thomas Owens	user1@example.com	127	2025-06-10 18:01:32	2025-06-10 18:57:32	20160.64	Debit Card
Thomas Owens	user1@example.com	135	2025-03-11 22:58:23	2025-03-11 23:11:23	8891.79	Debit Card
Thomas Owens	user1@example.com	144	2024-09-19 09:18:22	2024-09-19 10:10:22	12093.54	UPI
Thomas Owens	user1@example.com	221	2024-09-25 05:49:39	2024-09-25 06:25:39	12437.61	UPI
Thomas Owens	user1@example.com	278	2025-02-01 20:05:38	2025-02-01 21:05:38	32015.16	UPI
Thomas Owens	user1@example.com	379	2025-03-22 15:57:58	2025-03-22 16:48:58	21586.89	Credit Card
Charles Grant	user2@example.com	56	2024-06-23 15:59:57	2024-06-23 16:24:57	6828.61	Credit Card
Charles Grant	user2@example.com	169	2025-02-11 04:04:38	2025-02-11 04:37:38	9426.30	Credit Card
Charles Grant	user2@example.com	209	2024-10-16 10:07:48	2024-10-16 10:37:48	7690.62	Debit Card
Charles Grant	user2@example.com	223	2024-11-25 01:32:18	2024-11-25 02:09:18	41020.75	Debit Card

Result 41 ×

```
2180  19:23:07  SELECT c.name AS customer_name,  c.email AS email_address,  o.order_id,  o.order_date,  p.payment_date,  ...  400 row(s) returned
```

- Used for detailed transaction reports.

LEVEL 5 — Subqueries (Inner Queries)

Query: List all products priced above the average product price

```
2474    -- 1. Products priced above average price
2475 •  SELECT * FROM products
2476    WHERE price > (SELECT AVG(price) FROM products)
2477    ORDER BY price DESC;
```

product_id	name	category	price	stock_quantity	added_on
7	Response Indeed	Clothing	4897.36	36	2025-03-29 02:43:08
2	Population Social	Clothing	4813.68	84	2025-05-30 10:02:50
10	Development System	Electronics	4801.78	153	2025-03-12 08:22:57
4	Any Question	Clothing	4759.28	179	2025-06-03 13:34:03
35	Fire Often	Electronics	4734.89	198	2023-10-22 04:15:51
5	Natural Network	Toys	4722.66	75	2023-11-06 00:47:37
19	Build High	Clothing	4707.14	47	2023-09-01 02:50:01
41	Serious Recognize	Electronics	4523.10	112	2023-08-20 06:16:55
38	Study Total	Toys	4413.68	31	2023-11-08 08:25:07
20	Real Source	Books	4398.66	197	2025-02-08 10:28:27
33	Involve Officer	Clothing	4244.09	112	2024-03-22 11:36:17
48	Group But	Clothing	4180.23	141	2023-11-29 01:38:12
8	Every Amount	Home	4173.60	156	2025-04-30 03:11:10
22	Move Small	Books	4168.08	22	2023-11-12 18:26:30

products 42 ×

Apply

2181 19:37:09 SELECT * FROM products WHERE price > (SELECT AVG(price) FROM products) ORDER BY price DESC LIMIT 0, 1000 27 row(s) returned

- Used by pricing analysts to identify premium-priced products.

Query: Find customers who have placed at least one order

```
2479    -- 2. Active customers (who placed at least one order)
2480 •  SELECT name, email
2481    FROM customers
2482    WHERE customer_id IN (SELECT DISTINCT customer_id FROM orders);
```

name	email
Thomas Owens	user1@example.com
Charles Grant	user2@example.com
Kaitlin Richards	user3@example.com
Christina Williams	user4@example.com
David Allen	user5@example.com
Mark Duke	user6@example.com
Briana Wright	user7@example.com
John Bryan	user8@example.com
Jason Thompson	user9@example.com
Shawn Hill	user10@example.com
Walter Jenkins	user11@example.com
Mary Knight	user12@example.com
Leslie Wilson	user13@example.com
Deborah Arias	user14@example.com
Austin Flores	user15@example.com
Amy Landry	user16@example.com

customers 43 ×

2182 19:47:29 SELECT name, email FROM customers WHERE customer_id IN (SELECT DISTINCT customer_id FROM orders) LIMIT 0, 1000 30 row(s) returned

- Used to identify active customers for loyalty campaigns.

Query: Show orders whose total amount is above the average for that customer

```
2484    -- 3. Orders above customer's average order value
2485    SELECT o.*
2486    FROM orders o
2487    WHERE o.total_amount > (
2488        SELECT AVG(total_amount)
2489        FROM orders
2490        WHERE customer_id = o.customer_id
2491    );
```

order_id	customer_id	order_date	status	total_amount
6	29	2024-10-25 07:33:59	Cancelled	31050.17
8	19	2024-07-30 22:49:49	Cancelled	32714.06
9	6	2025-06-10 17:00:25	Pending	24219.20
10	28	2025-02-16 12:45:59	Delivered	24342.52
14	1	2024-09-24 21:21:38	Shipped	15803.34
16	5	2024-09-09 03:00:40	Shipped	22856.73
18	13	2024-06-15 12:57:04	Cancelled	32001.24
21	22	2025-03-12 18:53:46	Pending	25364.11
22	20	2025-02-21 11:21:06	Shipped	21281.44
24	6	2024-06-19 07:09:04	Pending	22882.19
25	17	2024-07-24 12:56:24	Pending	23749.66
26	24	2025-02-16 23:16:49	Delivered	35167.92
29	7	2025-02-17 18:03:18	Delivered	26308.25
30	23	2024-10-18 13:37:49	Delivered	18628.74
32	15	2024-10-20 06:47:38	Pending	23536.35
33	8	2025-04-27 01:26:11	Pending	36147.09

orders 44 × Apply

```
2183  19:50:06  SELECT o.* FROM orders o WHERE o.total_amount > (  SELECT AVG(total_amount)  FROM orders  WHERE customer...  194 row(s) returned
```

- Used to detect unusually high purchases per customer.

Query: Display customers who haven't placed any orders

```
2493    -- 4. Customers who never placed any order
2494    SELECT c.customer_id,
2495           c.name AS customer_name
2496    FROM customers AS c
2497    WHERE c.customer_id NOT IN (
2498        SELECT customer_id
2499        FROM orders
2500    );
```

customer_id	customer_name
NULL	NULL

```
2186  19:53:49  SELECT c.customer_id,  c.name AS customer_name FROM customers AS c WHERE c.customer_id NOT IN (  SELECT ...  0 row(s) returned
```

- Used for re-engagement campaigns targeting inactive users.

Query: Show products that were never ordered

```sql
-- 5. Products that were never ordered
SELECT *
FROM products
WHERE product_id NOT IN (
    SELECT product_id
    FROM orders
);
```

product_id	name	category	price	stock_quantity	added_on
NULL	NULL	NULL	NULL	NULL	NULL

2188 19:56:33 SELECT * FROM products WHERE product_id NOT IN (SELECT product_id FROM orders) LIMIT 0, 1000 0 row(s) returned

- Helps with inventory clearance decisions or product deactivation

Query: Show highest value order per customer

```sql
-- 6. Highest order value per customer
SELECT
    c.name,
    o.total_amount AS highest_order_value,
    o.order_date
FROM orders o
JOIN customers c ON o.customer_id = c.customer_id
WHERE o.total_amount = (
    SELECT MAX(total_amount)
    FROM orders
    WHERE customer_id = o.customer_id
)
ORDER BY o.total_amount DESC;
```

name	highest_order_value	order_date
Deborah Arias	48042.06	2025-02-01 02:02:18
Joseph Stuart	44504.56	2024-12-17 21:08:26
Kara Zavala	43867.08	2024-09-13 13:25:58
Charles Grant	42056.04	2025-01-27 15:23:27
Kaitlin Richards	41679.11	2024-11-08 07:40:55
Randy Mooney	40944.10	2025-06-08 16:15:49
Cindy Hart	40872.29	2024-12-17 18:53:05
Austin Flores	40510.54	2024-12-19 09:20:09
David Allen	39921.78	2025-03-21 21:25:00
Jeffrey Bray	39857.19	2025-04-13 20:12:26
Megan Lee	39563.51	2025-05-05 23:49:05
Mark Duke	39003.19	2025-01-10 15:50:45
Victoria Hall	38836.44	2025-04-18 05:53:20
Mary Knight	38656.26	2025-03-28 18:42:12
Amy Landry	37415.67	2024-11-21 16:50:01
Walter Jenkins	37129.82	2024-11-20 00:44:46

Result 50

2189 19:59:23 SELECT c.name, o.total_amount AS highest_order_value, o.order_date FROM orders o JOIN customers c ON o.custo... 30 row(s) returned

```sql
SELECT *
FROM orders o
WHERE total_amount = (
SELECT MAX(total_amount)
FROM orders
WHERE customer_id = o.customer_id
);
```

order_id	customer_id	order_date	status	total_amount
18	13	2024-06-15 12:57:04	Cancelled	32001.24
33	8	2025-04-27 01:26:11	Pending	36147.09
62	10	2025-06-13 23:21:01	Cancelled	35723.17
66	27	2024-06-19 09:41:16	Pending	26176.05
68	24	2025-03-15 07:27:20	Delivered	36159.92
80	12	2025-03-28 18:42:12	Delivered	38656.26
107	14	2025-02-01 02:02:18	Shipped	48042.06
121	16	2024-11-21 16:50:01	Pending	37415.67
128	7	2025-05-05 16:49:18	Pending	28589.04
133	11	2024-11-20 00:44:46	Shipped	37129.82
139	18	2025-04-13 20:12:26	Delivered	39857.19
174	19	2024-09-30 11:32:38	Cancelled	35676.00
189	25	2024-12-17 18:53:05	Shipped	40872.29
196	21	2025-02-07 10:13:15	Shipped	24096.64
198	22	2024-11-11 00:17:56	Delivered	36504.68
243	15	2024-12-19 09:20:09	Delivered	40510.54

orders 51 ×

Apply

2190 20:01:41 SELECT * FROM orders o WHERE total_amount = (SELECT MAX(total_amount) FROM orders WHERE customer_id = o.custo... 30 row(s) returned

- Used by pricing analysts to identify premium-priced products

Query: Highest order per customer with customer name

```sql
-- 7.  Highest Order Per Customer
SELECT c.name,
       o.order_id,
       o.total_amount
FROM orders o
JOIN customers c
    ON o.customer_id = c.customer_id
WHERE o.total_amount = (
    SELECT MAX(total_amount)
    FROM orders
    WHERE customer_id = o.customer_id
);
```

name	order_id	total_amount
Thomas Owens	278	32015.16
Charles Grant	315	42056.04
Kaitlin Richards	281	41679.11
Christina Williams	386	25747.34
David Allen	266	39921.78
Mark Duke	330	39003.19
Briana Wright	128	28589.04
John Bryan	33	36147.09
Jason Thompson	348	21414.44
Shawn Hill	62	35723.17
Walter Jenkins	133	37129.82
Mary Knight	80	38656.26
Leslie Wilson	18	32001.24
Deborah Arias	107	48042.06
Austin Flores	243	40510.54
Amy Landry	121	37415.67

Result 52 ×

```
2191  20:05:22  SELECT c.name,    o.order_id,    o.total_amount FROM orders o JOIN customers c   ON o.customer_id = c.customer_id ...   30 row(s) returned
```

- Used to identify the largest transaction made by each customer. Outputs name as well.

LEVEL 6 — Set Operations

Query: Customers who placed an order OR wrote a review

```
2545    -- Level 6: Set Operations
2546
2547    -- 1. Customers who either placed order OR gave review
2548    SELECT name, email FROM customers
2549    WHERE customer_id IN (
2550        SELECT customer_id FROM orders
2551        UNION
2552        SELECT customer_id FROM product_reviews
2553    )
2554    ORDER BY name;
```

name	email
Adrienne Green	user27@example.com
Amanda Bright	user19@example.com
Amy Landry	user16@example.com
Austin Flores	user15@example.com
Brandy Wright	user24@example.com
Briana Wright	user7@example.com
Charles Grant	user2@example.com
Christina Williams	user4@example.com
Cindy Hart	user25@example.com
David Allen	user5@example.com
Deborah Arias	user14@example.com
Jason Thompson	user9@example.com
Jeffrey Bray	user18@example.com
Jessica Zamora	user21@example.com
John Bryan	user8@example.com
Joseph Stuart	user28@example.com

customers 53 ×

```
2192  20:11:11  SELECT name, email FROM customers  WHERE customer_id IN (    SELECT customer_id FROM orders    UNION    SELECT...   30 row(s) returned
```

- Used to identify engaged customers from different activity areas.

Query: Customers who placed an order AND wrote a review

```
2556    -- 2. Highly engaged customers (placed order AND gave review)
2557    SELECT DISTINCT c.name, c.email
2558    FROM customers c
2559    WHERE EXISTS (SELECT 1 FROM orders o WHERE o.customer_id = c.customer_id)
2560      AND EXISTS (SELECT 1 FROM product_reviews r WHERE r.customer_id = c.customer_id)
2561    ORDER BY c.name;
```

name	email
Adrienne Green	user27@example.com
Amanda Bright	user19@example.com
Austin Flores	user15@example.com
Briana Wright	user7@example.com
Charles Grant	user2@example.com
Christina Williams	user4@example.com
Cindy Hart	user25@example.com
Deborah Arias	user14@example.com
Jason Thompson	user9@example.com
Jessica Zamora	user21@example.com
Joseph Stuart	user28@example.com
Kara Zavala	user29@example.com
Kathryn Mathews	user23@example.com
Leslie Wilson	user13@example.com
Mark Duke	user6@example.com
Megan Lee	user20@example.com

customers 54 ×

2193 20:14:13 SELECT DISTINCT c.name, c.email FROM customers c WHERE EXISTS (SELECT 1 FROM orders o WHERE o.customer_id ... 23 row(s) returned

- Used to identify highly engaged customers for rewards.

Key Business Insights

Revenue Performance:

The retail business generated total revenue of approximately **₹69 Lakhs+** across **400+ customer orders**, indicating strong sales activity and consistent customer demand.

Customer Behaviour:

Analysis of customer transactions shows that approximately **30 customers are actively purchasing products**, with varying order frequencies and spending patterns. This information can be used to identify loyal customers and design targeted marketing campaigns.

Product Performance:

Among all product categories, **Clothing** and **Electronics** emerged as the highest-performing categories in terms of sales volume and customer demand. These categories contribute significantly to overall business revenue.

Payment Preferences:

Customers predominantly prefer digital payment methods such as **Debit Card**, **Credit Card**, and **UPI**, highlighting the growing adoption of cashless transactions in retail operations.

Inventory Alerts:

Inventory analysis revealed that several products are either **out of stock** or have **low stock levels**, emphasizing the need for timely inventory replenishment to avoid potential loss of sales opportunities.

Customer Engagement:

A significant number of customers not only purchase products but also provide product reviews and ratings. This indicates strong customer engagement and provides valuable feedback for product improvement and customer satisfaction analysis.

Conclusion

The **Retail Store Database Management System** project successfully demonstrates the practical application of SQL in managing and analysing retail business data. The project covers the complete database lifecycle, including database design, table creation, data insertion, data retrieval, aggregation, joins, subqueries, and set operations.

The database structure is well-organized and follows relational database principles, enabling efficient storage and retrieval of information related to customers, products, orders, payments, and reviews. The SQL queries developed throughout the project provide meaningful business insights that can support strategic decision-making.

The findings from this analysis can help businesses improve inventory management, identify valuable customers, optimize pricing strategies, monitor sales performance, and enhance customer engagement.

Overall, this project strengthened practical database management skills and demonstrated how SQL can be effectively used for real-world business analysis.